

Build Your VPC and Launch a Web Server

A complete step-by-step walkthrough of AWS Academy Lab 2 – covering both the **Curriculum Guided Track** and the **Sandbox Learner Lab Track**. By the end of these labs, you will have built a fully functional Virtual Private Cloud, configured public and private subnets, attached an Internet Gateway, and deployed a live web server on AWS.

AWS ACADEMY LAB 2

GUIDED + SANDBOX TRACKS



Lab Overview: Two Learning Tracks

This tutorial covers two distinct lab environments designed for different stages of learner readiness. Each track builds core AWS networking competency.

Track 1: Guided Lab

Pre-packaged, step-by-step instructions inside the AWS Academy Curriculum environment. Ideal for first-time learners following a structured path with automatic grading checkpoints.

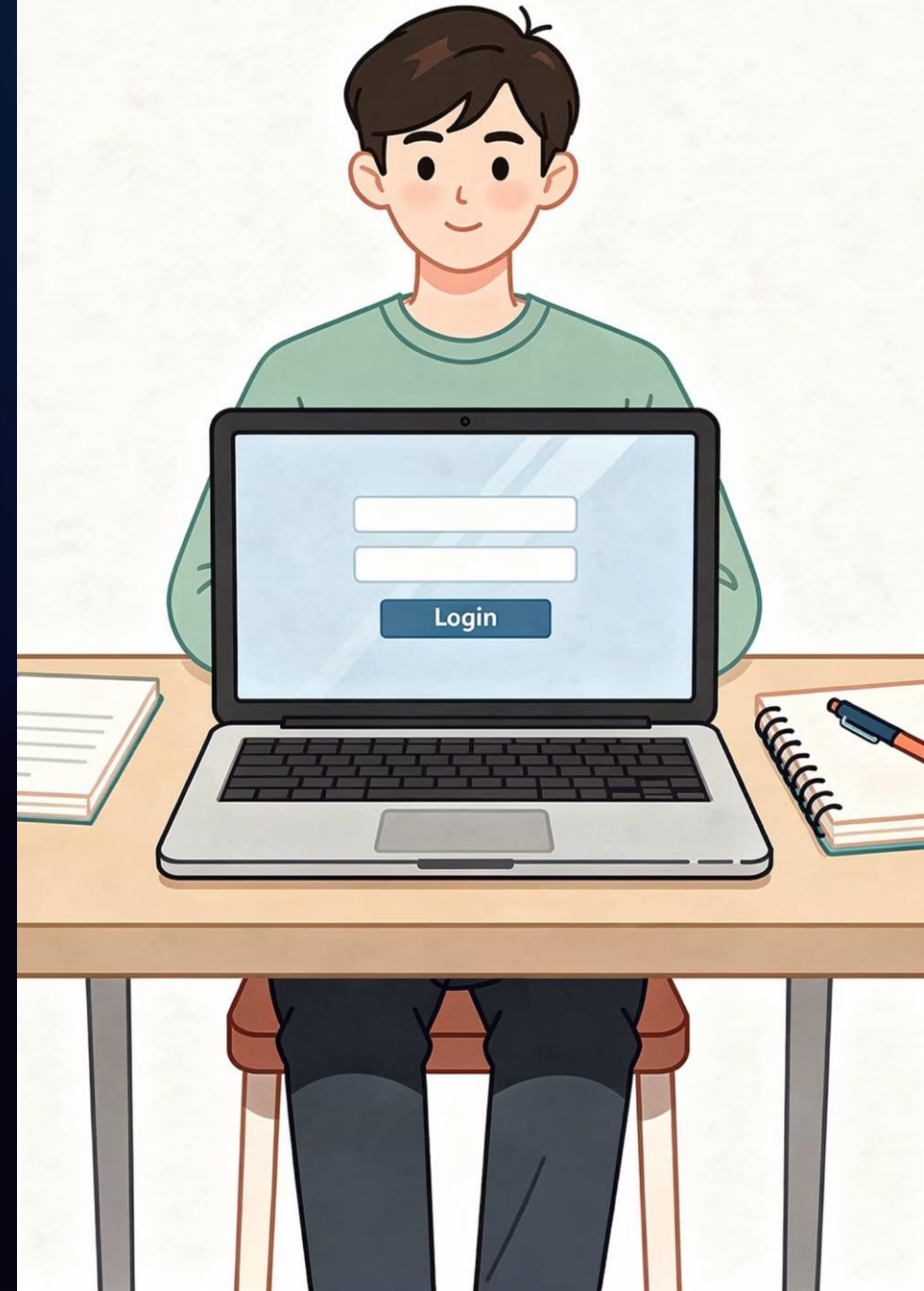
Track 2: Sandbox Lab

Unguided, from-scratch architecture design inside the AWS Academy Learner Lab. Designed for learners ready to engineer a secure two-tier VPC independently within a \$100 budget.

Part 1: Guided Lab

Curriculum Pre-Packaged Track

Section 6 — AWS Academy Lab 2: Build Your VPC and Launch a Web Server using the official guided curriculum environment.



Section 6.1: Lab Prerequisites

Before beginning Lab 2, confirm that all required infrastructure elements and access credentials are in place. Operating in the correct environment prevents score desynchronization and access errors.

AWS Academy Learner Dashboard

Log in via your institutional portal. Verify your course enrollment and that Lab 2 is visible under the Modules tab before proceeding.

Private / Incognito Browser Window

Always open the lab in a private browser session. This prevents cached credentials or cookies from interfering with the AWS Management Console authentication flow.

Pre-Provisioned Lab Parameters

The lab environment auto-supplies all required network asset configuration parameters. You do not need to create these manually – they are injected by the background lab template engine.

Critical Curriculum Constraints

Violating these constraints will trigger immediate **Access Denied** errors and may desynchronize your lab tracking score. Read carefully before starting.

Do NOT Create Custom IAM Users or Groups

The restricted AWS Academy Student profile blocks IAM User, IAM Group, and root organizational policy changes. Attempting these actions generates an Access Denied error that breaks your lab score tracker.

Always Attach LabRole

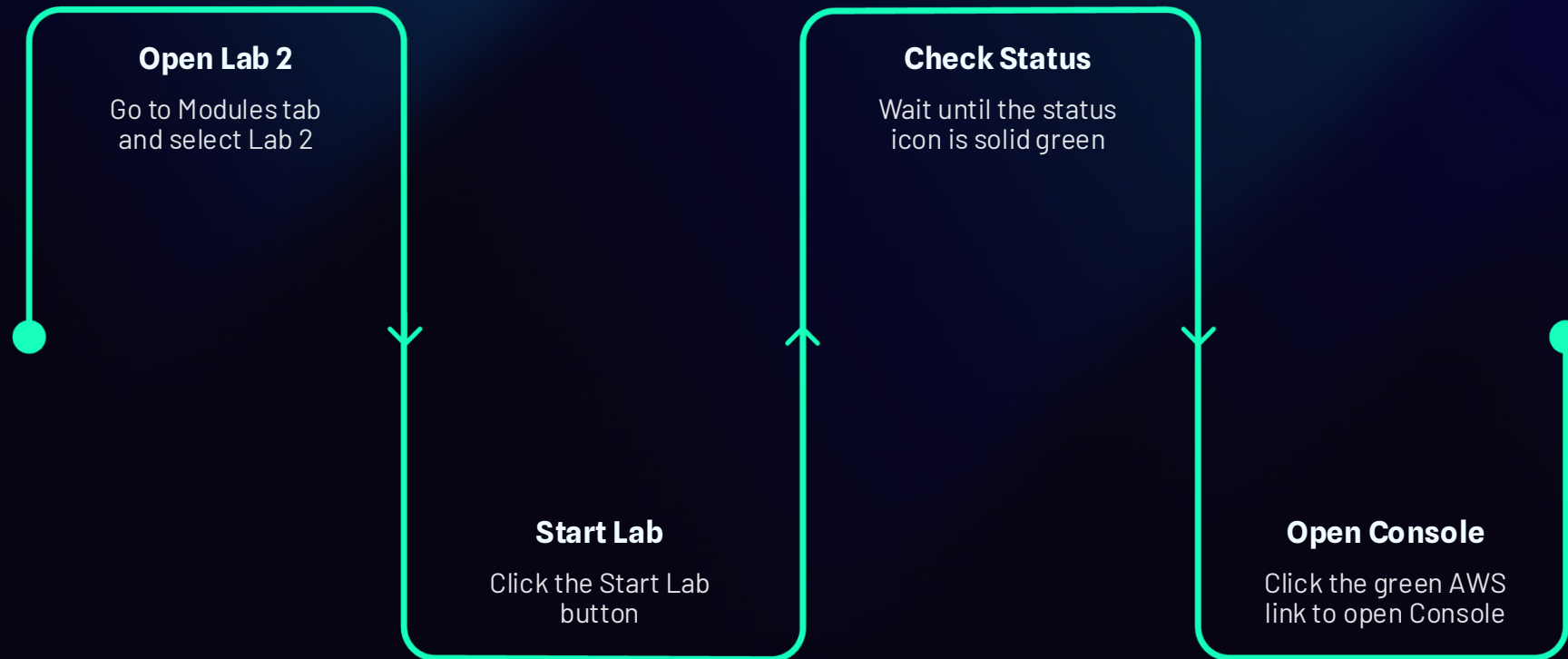
When launching instances, configuring access controls, or authorizing infrastructure permissions, you **must** select the pre-configured **LabRole** IAM instance profile. This is a mandatory critical step throughout the lab.



Missing the LabRole attachment step will cause your instance to fail validation — always double-check before launching.

Task 1: Access the Lab Sandbox

These steps authenticate you into the live AWS Management Console through the Academy portal. Complete each step in sequence — do not skip the status indicator check.



The status indicator transitions from **blinking amber** to **solid green** when the sandbox environment is fully provisioned and ready. Clicking the AWS link before it turns green may result in a partial or failed console session.

Task 2: Build a Custom VPC

You will create the core network boundary – **Lab-VPC** – using a manually entered IPv4 CIDR block. This VPC will contain all resources built throughout the remainder of the lab.

VPC Configuration Parameters

Resource Type: VPC only

Name tag: Lab-VPC

IPv4 CIDR input mode: Manual

IPv4 CIDR block: 10.0.0.0/16

How to Navigate There

1. Type **VPC** in the top global search bar and select the VPC service.
2. In the left navigation pane, select the **VPC** dashboard link.
3. Click the orange **Create VPC** button at the top right.
4. Select the **VPC only** radio button option.
5. Enter the parameters shown, then click **Create VPC**.

Task 2: Create Public & Private Subnets

After creating the VPC, you will add two subnets — one for publicly accessible resources and one for isolated backend components. Both subnets reside in the same Availability Zone.

Public Subnet 1

Subnet name: Public Subnet 1

Availability Zone: First zone in list (e.g., us-east-1a)

IPv4 CIDR block: 10.0.0.0/24

This subnet will host the web server instance with a public IP address and internet access.

Private Subnet 1

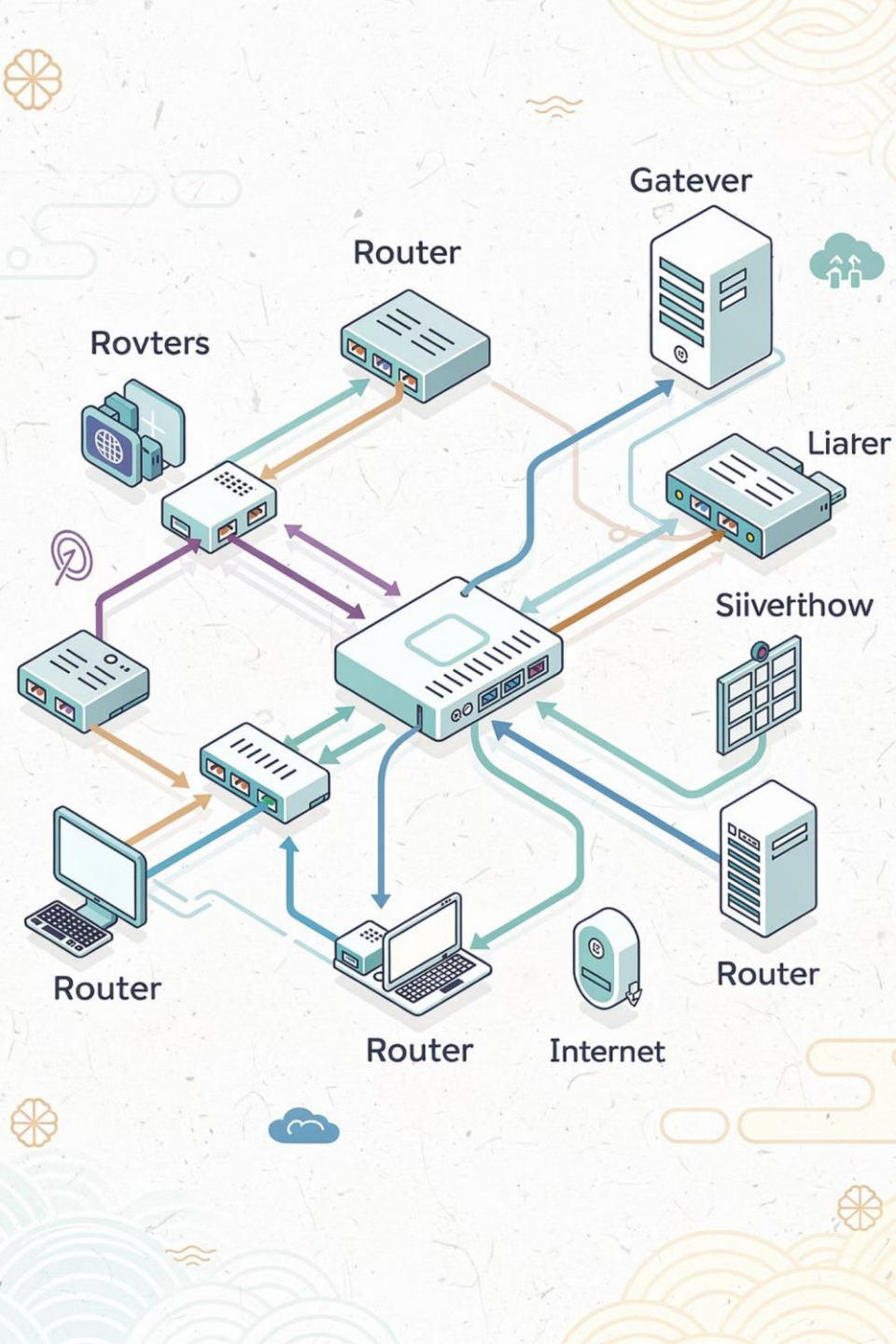
Subnet name: Private Subnet 1

Availability Zone: Same zone as Public Subnet 1

IPv4 CIDR block: 10.0.1.0/24

This isolated subnet will host backend resources that should not be directly reachable from the internet.

Navigate to **Subnets** in the left pane, click **Create subnet**, select **Lab-VPC** from the VPC ID dropdown, and configure both subnets using **Add new subnet** before clicking Create.



Task 3: Attach an Internet Gateway

An Internet Gateway (IGW) enables communication between your VPC and the public internet. Without it, even publicly addressed instances cannot send or receive external traffic.

1

Create the IGW

In the left pane, select **Internet gateways**. Click **Create internet gateway** and enter the Name tag: **Lab-IGW**. Click Create.

2

Attach to VPC

After creation, locate the green success banner and click the **Actions** dropdown. Select **Attach to VPC**.

3

Select Lab-VPC

In the Available VPCs list, choose **Lab-VPC** and click **Attach internet gateway** to complete the connection.

Task 4: Configure Routing & Public Access

The route table defines how traffic flows in and out of your subnets. You will add a default route pointing to the Internet Gateway and associate it with your public subnet only.



To locate the correct route table, look for the one bound to **Lab-VPC** in the Route tables view. You may rename it **Public Route Table** for clarity. **Do not** associate Private Subnet 1 with this route table — doing so would expose your backend tier to the public internet.

Task 5: Launch a Web Server Instance

With the network fully configured, you will now deploy an EC2 instance into the public subnet. This instance will serve as your web server, running a pre-built PHP application via a startup script.

1

Instance Identity

Name: Web Server 1

AMI: Amazon Linux 2023

Instance type: t2.micro

Key pair: Proceed without a key pair

2

Network Placement

VPC: Lab-VPC

Subnet: Public Subnet 1

Auto-assign public IP: Enable

3

Security Group

Name: Web-SG

Description: Allow HTTP traffic

Inbound rule: HTTP / Anywhere-IPv4 (0.0.0.0/0)

4

Advanced Details

IAM instance profile: LabRole (CRITICAL)

User data: Paste full bootstrap script (see next card)

User Data Bootstrap Script

Paste this script exactly into the **User data** field in the Advanced details panel before launching the instance. It installs Apache, PHP, MariaDB, and deploys the lab web application automatically on first boot.

```
#!/bin/bash
dnf install -y httpd wget php mariadb105-server
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/\
CUR-TF-100-ACCLFO-2/2-lab2-vpc/s3/lab-app.zip
unzip lab-app.zip -d /var/www/html/
chkconfig httpd on
service httpd start
```

 Copy the script in its entirety. Any missing line will prevent the web server from starting correctly, causing the validation step to fail.

Section 6.3: Validation & Submission

After launching the instance, complete these verification steps to confirm your architecture is correctly configured before submitting for grading.

01

Confirm Status Checks Pass

In the EC2 Instances view, wait until **Web Server 1** shows **2/2 checks passed**. This confirms the instance is healthy and network connectivity is established.

03

Test in a Private Browser Tab

Paste the DNS address into a new **private/incognito** browser window and press Enter. The lab web application should load, displaying the AWS logo and live instance metadata.

02

Copy the Public IPv4 DNS

Click the instance row, open the lower details pane, and copy the **Public IPv4 DNS** address string in full.

04

Submit for Grading

Return to the AWS Academy instruction pane and click the **Submit** button. The automated checkpoint script will validate your environment and record your lab grade.

Guided Lab: Validation Milestone

2/2 Checks Passed

When the web application loads and displays the AWS logo alongside live instance metadata, your route table, Internet Gateway, security group, and EC2 configuration are all correct. You are ready to submit.



Part 2: Sandbox Lab

Unguided Learner Track

Section 7 – Engineering an Isolated Two-Tier Network Environment from scratch inside the AWS Academy Sandbox with a \$100 budget limit.



Section 7.1: Sandbox Lab Prerequisites

The Sandbox Lab runs in an open-ended environment. Unlike the Guided Lab, there are no pre-provisioned parameters — you design and deploy the entire architecture yourself within defined budget and policy boundaries.



Sandbox Environment Access

Log in to your AWS Academy portal and navigate to the **Learner Lab** section. Start the lab and wait for the green status indicator before accessing the console.



Text Editor for Tokens

Keep a plain text editor open throughout the lab. You will need to record dynamically generated resource IDs, Elastic IP addresses, and gateway identifiers as you build.



Console or CLI Access

Use either the AWS Management Console web interface or the AWS CLI. Both provide full access to the services required for this architecture build.

Sandbox Cost Controls & Boundaries

The Sandbox Lab allocates a **\$100 budget limit** per student. Exceeding this limit or leaving resources running after the lab will deplete your balance and may affect your ability to complete future labs.

No Premium Features

Do not use advanced security suites, large compute options, or premium paid add-ons. Stick strictly to the services specified in the lab instructions.

Free-Tier Instance Shapes Only

Always select **t2.micro** or **t3.micro** instance types. These fall within free-tier limits and protect your \$100 sandbox balance from unexpected compute charges.

Clean Up Immediately After Validation

Execute the full deconstruction sequence as soon as you finish validation. NAT Gateways and Elastic IPs continue billing even when idle — failing to release them causes ongoing charges.

Task 1: Initialize the Sandbox Environment

Starting the Sandbox Lab provisions a clean AWS account workspace separate from the Guided Lab. This workspace has broader permissions but operates under Service Control Policies (SCPs) and the \$100 budget cap.

Step-by-Step

1. Log in to your AWS Academy portal home screen.
2. Navigate to the **Learner Lab** section.
3. Click the **Start Lab** button.
4. Watch the status indicator. Wait until the circle turns **solid green**.
5. Click the **AWS Console** link to launch your sandbox workspace.

What to Expect

The sandbox workspace launches as a clean, empty AWS account. No pre-configured resources exist — you will build everything from scratch. The environment persists between sessions as long as your budget is not exhausted.

Record your session start time and any resource IDs you create in your text editor as you proceed through each task.

Task 2: Build a Two-Tier VPC from Scratch

In the Sandbox Lab, you use the **VPC and more** wizard option, which automatically creates subnets, route tables, and an Internet Gateway in a single guided flow – saving time while still allowing customization.

VPC Wizard Parameters

Configuration mode: VPC and more

Name project prefix: Sandbox-Net

IPv4 CIDR block: 192.168.0.0/16

Availability Zones: 1

Public Subnets: 1

Private Subnets: 1

Verify Subnet CIDR Allocation

Expand the Subnet CIDR blocks panel after entering parameters to confirm the wizard assigned:

Public subnet: 192.168.0.0/24

Private subnet: 192.168.1.0/24

NAT & Endpoints: Set to None

Set both **NAT Gateways** and **VPC Endpoints** to **None**. You will configure the NAT Gateway manually in Task 3 to control costs and understand the provisioning process.

Task 3: Configure an Outbound-Only NAT Gateway

A NAT Gateway allows instances in the **private subnet** to initiate outbound internet connections (e.g., software updates) while blocking any inbound connections from external sources – providing secure outbound-only access.

⊗ **CRITICAL:** Place the NAT Gateway inside the **Public Subnet**, NOT the Private Subnet. Placing it in the wrong subnet will break outbound routing for your private tier.

1

Verify IGW Exists

In the left pane, confirm that **Sandbox-Net-igw** was auto-created and attached to your VPC by the wizard.

2

Create NAT Gateway

Name: Sandbox-NAT-Gateway

Subnet: Public Subnet (Sandbox-Net-subnet-public)

Connectivity type: Public

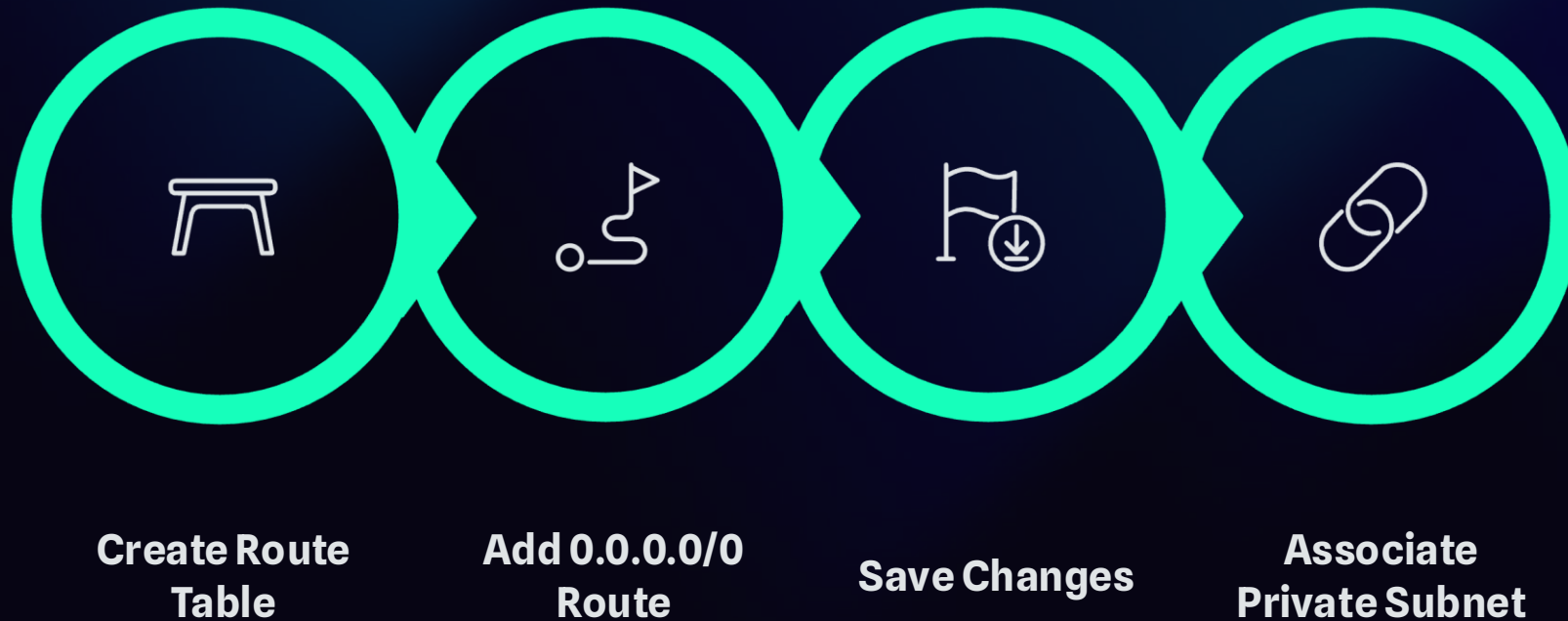
3

Allocate Elastic IP

Click **Allocate Elastic IP** to assign a static public IP from the AWS pool. Then click **Create NAT gateway**. Allow up to 2 minutes for provisioning.

Task 4: Engineering Private Subnet Routing

The private subnet needs its own route table that directs all outbound traffic through the NAT Gateway rather than the Internet Gateway — ensuring private instances can reach the internet without being directly exposed to it.



Navigate to **Route tables** in the left pane. After creating **Private-Route-Table** and adding the NAT Gateway route, switch to the **Subnet associations** tab and check only **Sandbox-Net-subnet-private**. Save associations to complete the routing configuration.

Task 5: Implement Stateful Instance-Level Firewalls

Security groups act as stateful virtual firewalls at the instance level. In this task you create a **least-privilege** security group that allows only your public subnet's application tier to reach database ports — blocking all external IP addresses.

Security Group Settings

Name: App-DB-SG

Description: Restricted database port filter

VPC: Sandbox-Net

Inbound Rule Configuration

Type: MySQL/Aurora (Port 3306) or MS SQL (Port 1433) — match your database variant

Source type: Custom

Source CIDR: 192.168.0.0/24 (Public Subnet only)

This rule restricts database access exclusively to frontend application instances in the public subnet, blocking all external IPs from scanning or connecting to the database port.

Section 7.3: Structural Validation

Launch a test instance into the private subnet to verify your network isolation is working as designed. A correctly isolated instance will have a private IP address only – no public IP should be assigned.



Validation: Launch a Private Test Instance

This test instance — **SandboxStorageHost** — validates that your private subnet, security group, and IAM role configuration are all functioning correctly before you submit.

01

Navigate to EC2 & Launch Instance

Go to the EC2 Dashboard and click **Launch instances**. Name the instance **SandboxStorageHost** and select an approved free-tier **t2.micro** AMI.

03

Attach Security Group & LabRole

Select the existing **App-DB-SG** security group. In Advanced details, attach the **LabRole** IAM instance profile. Click **Launch instance**.

02

Map to Private Subnet

Under Network settings, click **Edit**. Select **Sandbox-Net** as the VPC and choose the **Private Subnet** as the target. Set **Auto-assign public IP** to **Disable**.

04

Confirm Validation Checkpoint

In the instance details pane, verify the instance shows a **private IP address only** with no public IP mapping. This confirms your network isolation architecture is functioning correctly.

Validation Checkpoint: What to Look For

Inspect the instance details panel after **SandboxStorageHost** finishes initializing. The presence or absence of certain fields confirms whether your architecture is correct.

Expected: Private IP Address

The instance should display an internal private IP address in the **192.168.1.x** range, confirming correct placement inside the private subnet.

Expected: No Public IP

The Public IPv4 address field should be **empty or absent**. Any public IP assignment indicates the instance was incorrectly placed in the public subnet or auto-assign was not disabled.



No public IP on the private instance = your two-tier network isolation is working correctly. 



Section 7.4: Environment Deconstruction

Clean Up Now

Execute the full teardown sequence immediately after validation. NAT Gateways and Elastic IPs generate ongoing charges even when idle – every minute counts against your \$100 balance.

Step-by-Step Teardown Sequence

Follow this exact order. The VPC deletion step should be performed **last** — the cascading teardown engine will automatically remove subnets, security groups, and gateway attachments concurrently.

1 Terminate SandboxStorageHost

In the EC2 Instances dashboard, select **SandboxStorageHost**, click **Instance state** → **Terminate instance**. Wait until the status reads **terminated** before proceeding.

2 Delete the NAT Gateway

In the VPC Dashboard, go to **NAT Gateways**, select **Sandbox-NAT-Gateway**, click **Actions** → **Delete NAT Gateway**. Wait for deletion to complete.

3 Release the Elastic IP

Go to **Elastic IPs**, select the IP allocated for the NAT Gateway, click **Actions** → **Release Elastic IP addresses**. This stops idle reservation charges immediately.

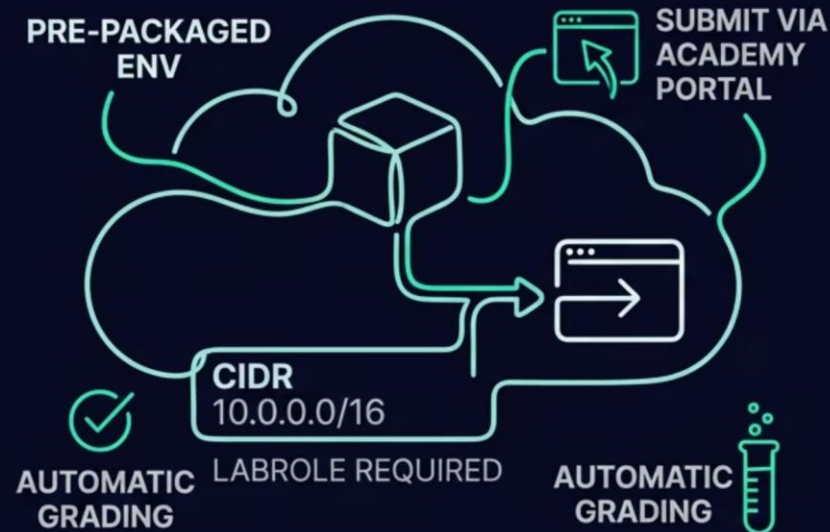
4 Delete the VPC

Select **Sandbox-Net VPC** (or Custom-Net-VPC), click **Actions** → **Delete VPC**. The automated resource engine executes a cascading teardown of all dependent resources.

Architecture Comparison: Guided vs. Sandbox

Both labs build VPC-based architectures, but differ significantly in scope, configuration approach, and learning outcomes.

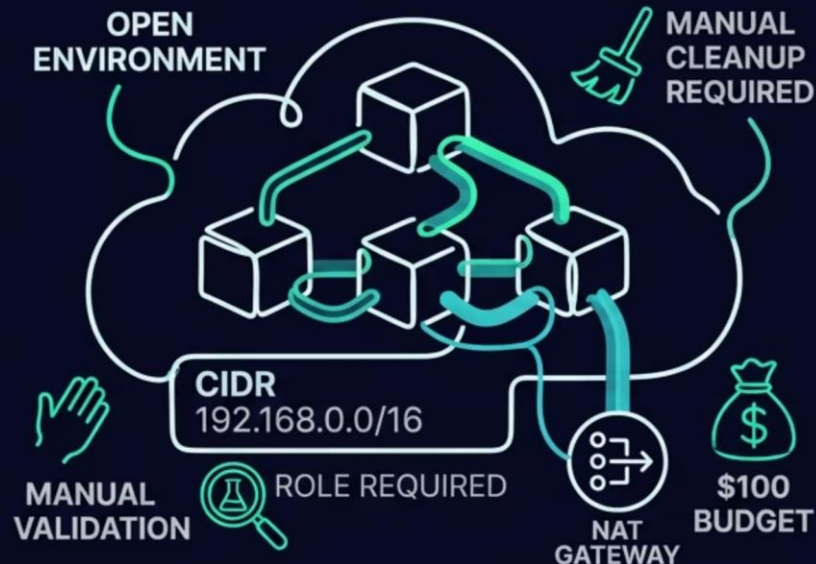
GUIDED LAB



**SIMPLE PUBLIC
ROUTING**

**NO NAT
GATEWAY**

SANDBOX LAB



**TWO-TIER
ISOLATION**

**NAT
GATEWAY**

Core Networking Concepts Covered

These labs build foundational AWS networking knowledge that applies directly to real-world cloud architecture design.



Virtual Private Cloud (VPC)

A logically isolated section of the AWS Cloud where you define your own IP address space, subnets, and routing configuration using CIDR notation.



Public vs. Private Subnets

Public subnets route traffic through an Internet Gateway for external access. Private subnets use a NAT Gateway for outbound-only connectivity, protecting backend resources from direct exposure.



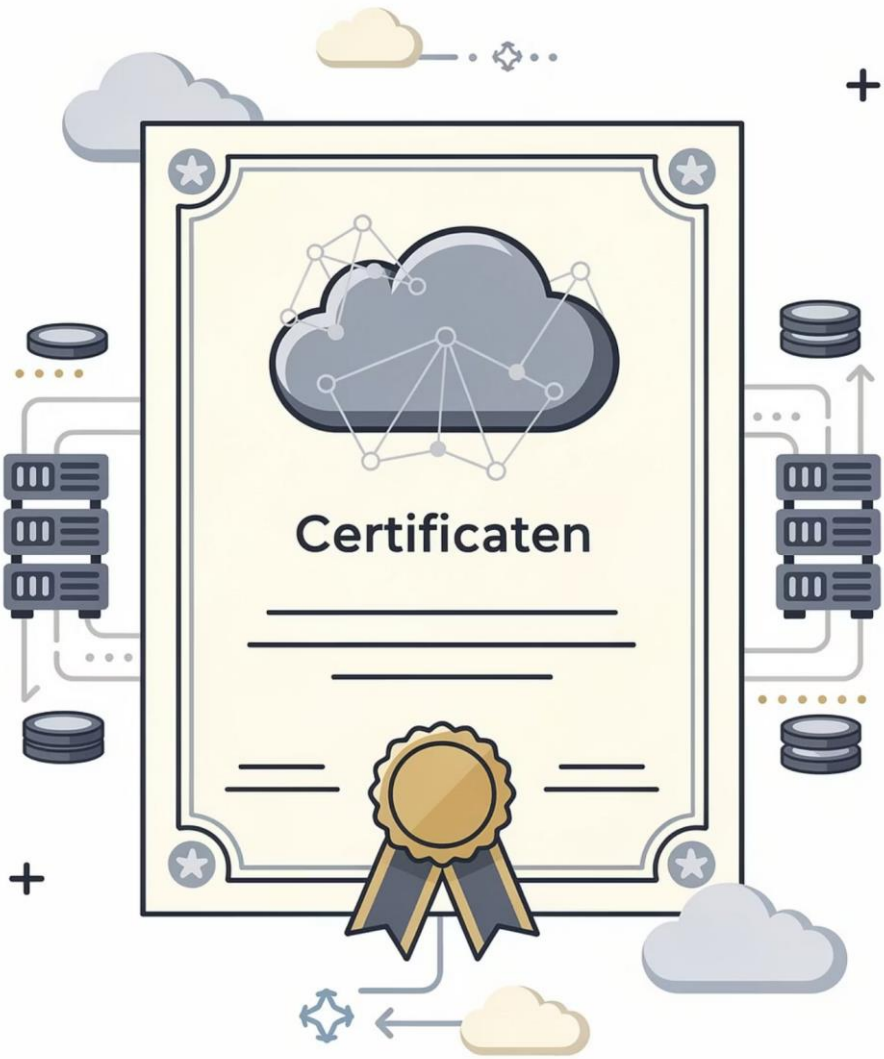
Security Groups & Least Privilege

Stateful instance-level firewalls that enforce least-privilege access – restricting database port access to the application subnet CIDR only, blocking all external IPs by default.



Route Tables & Internet Gateway

Route tables define traffic flow within and beyond the VPC. An Internet Gateway enables full bidirectional internet access for public subnets. A NAT Gateway provides outbound-only access for private subnets.



Lab Complete: Key Takeaways

You have successfully built a functional two-tier VPC architecture on AWS — from CIDR configuration through web server deployment and network isolation validation.

Guided Lab

VPC + 2 subnets + IGW + route table + EC2 web server deployed and validated via public DNS. Lab grade submitted through the Academy portal.

Sandbox Lab

Two-tier isolated VPC with NAT Gateway, private route table, least-privilege security group, and a private-only EC2 instance validated and torn down cleanly.

Always Remember

Attach **LabRole** to every instance. Clean up Sandbox resources immediately. Never create custom IAM users in the Guided Lab environment.